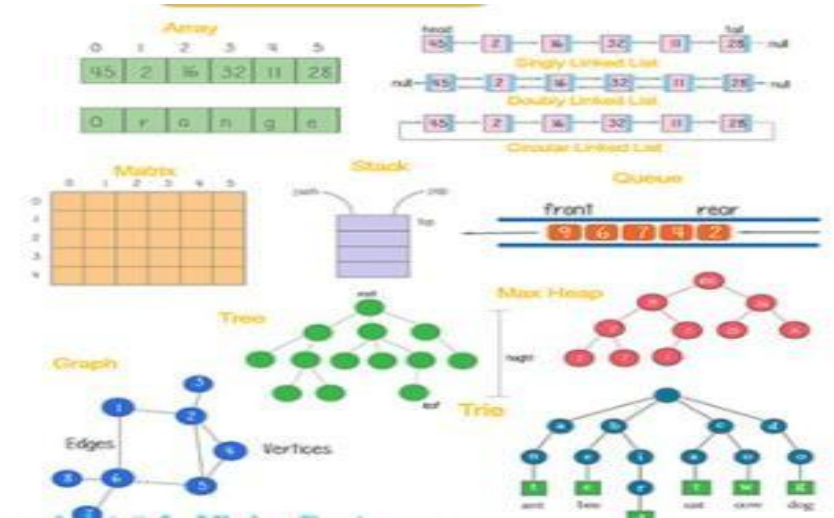


Data Structures

Dr. Elakkiya E
Asst. Professor,
Dept. of CSE, SRM University AP



SRM
UNIVERSITY AP
— Andhra Pradesh

NOTE: The syllabus given herein shall be followed and further additions/deletions are possible based on the time factor.

❑ Unit-I

- Introduction to data structures, Abstract Data Type (ADT) representation and implementation of ADT, time and space requirements of algorithms.
- Array ADT, representing polynomials, sparse matrices using arrays and their operations.
- Stacks and Queues: Representation and application, implementation of stack and queue operations.

❑ Unit-II

- Linked lists: Single linked lists, implementation of link list and various operation, Double linked list and circular list.

❑ Unit-III

Trees: Tree terminology, Binary tree, Binary search tree, Complete Binary Tree representation, infix to post fix conversion, postfix expression evaluation. General tree, AVL Tree.

❑ Unit-IV

Graphs: Graph terminology, Representation of graphs, Path matrix, BFS (breadth first search), DFS (depth first search), topological sorting, Shortest path algorithms

❑ Unit-V

Sorting and Searching techniques:

- Bubble sort, Insertion sort, Quick sort, Merge sort, Heap sort, Radix sort, Heaps & Priority Heaselection sortps and Binomial Heaps
- Linear Search and Binary Search methods, Hashing techniques and hash functions.

Text Books and other references

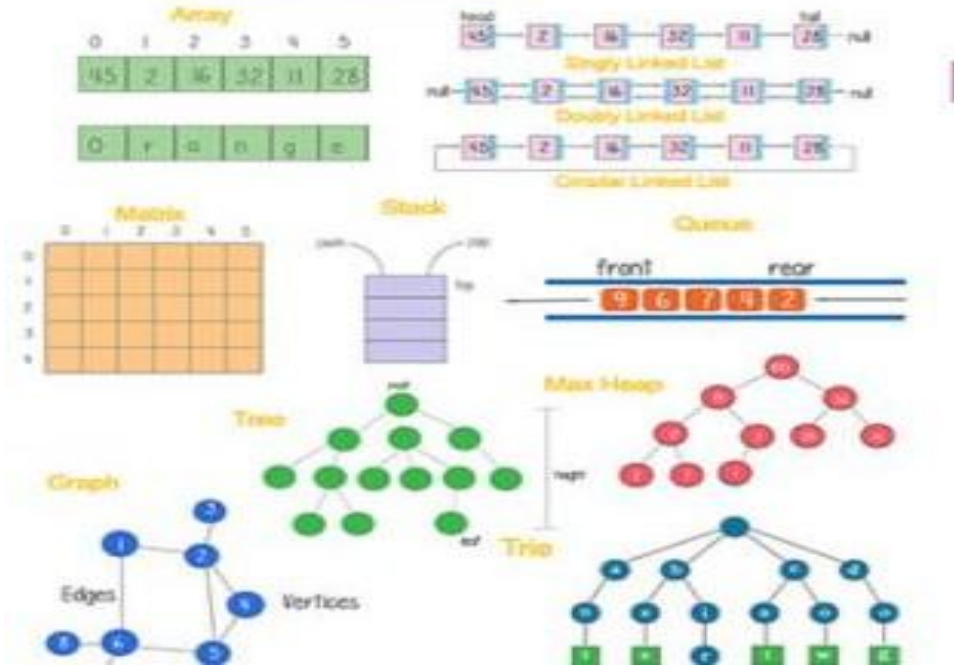
► Books of Study:

1. *“Data Structures A Pseudo Approach with C”*, Richard F. Gilberg & Behrouz A. Forouzan, Cengage learning, 2nd edition.
2. *“Data structures using C”*, Aaron M. Tenenbaum, Y Langsam and Mosche J. Augenstein, Pearson publication.

► Books of References:

1. *“Fundamentals of data structure in C”*, Horowitz, Sahani & Anderson Freed, Computer Science Press.
2. *“Data Structures Using C”*, Reema Thareja, Oxford Higher Education, 2nd edition.
3. *“Fundamental of Data Structures”*, (Schaums Series) Tata-McGraw-Hill.
4. *“Data Structures & Algorithms; Concepts, Techniques & Algorithms”*, G. A. V. Pai, TMG Hill.

Data structures



INTRODUCTION TO

DATA STRUCTURES

Outline of Unit-I

Data Structures

Unit-I

Unit-II

Unit-III

Unit-IV

Unit-V

1. Introduction to Data Structures

- ▶ Why Data Structures?
- ▶ What is Data Structure?
- ▶ Types of Data Structures

2. Abstract Data Types (ADT)

- ▶ Why ADT?
- ▶ What is ADT?
- ▶ ADT implementation

3. Algorithm Efficiency

- ▶ What is an Algorithm?
- ▶ Pseudocode?
- ▶ Complexity Analysis

- Big-O notation
- Time Complexity
- Space Complexity

4. Applications of Array ADT

- ▶ Sparse matrix representation & implementation
- ▶ **Polynomial Expressions representation & implementation**

5. Stack ADT

- ▶ Stack representation
- ▶ Array implementation of Stacks
- ▶ Applications of Stacks

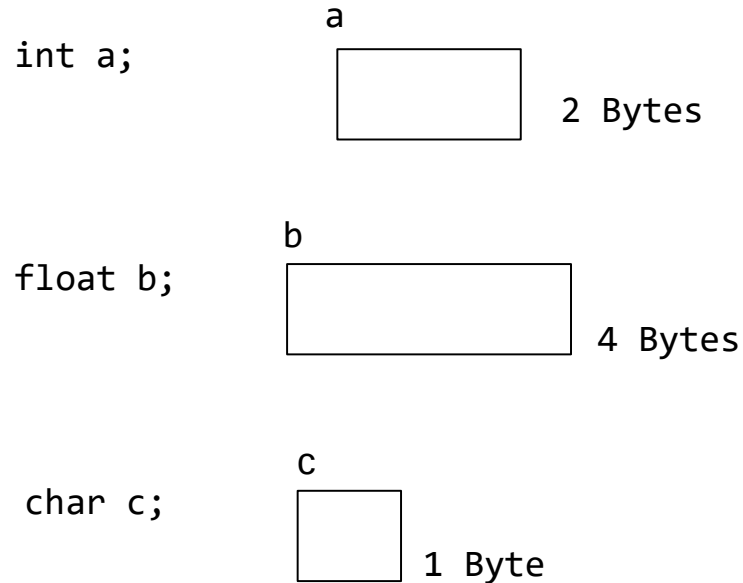
- Infix, prefix, postfix Conversions
- Evaluation of Postfix and Prefix expressions

6. Queue ADT

- ▶ Queue representation
- ▶ Array implementation of Queues
- ▶ Circular Queues
- ▶ Applications of Queues

Why Data Structures?

What is a variable?



A variable is a name of the **memory location**.

What is a Data Type?

Type	Values	Operations
integer	$-\infty, \dots, -2, -1, 0, 1, 2, \dots, \infty$	$*, +, -, \%, /, ++, --, \dots$
floating point	$-\infty, \dots, 0.0, \dots, \infty$	$*, +, -, /, \dots$
character	$\backslash 0, \dots, 'A', 'B', \dots, 'a', 'b', \dots, \sim$	$<, >, \dots$

A Data Type consists of two parts: **a set of data**, and the **operations** that can be performed on the data.

Why Data Structures?

○ What is Data?

Collections of facts from which conclusions **may be** drawn is known as data.

○ What is an Information?

Data from which **decisions** can be made is known as information.

► Atomic Data

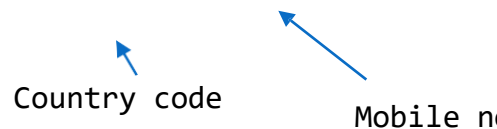
The data that consists of single piece of information.

```
int a=2314;  
(Data can not be divided further)
```

► Composite Data

The data that can be divided (not an atomic data)

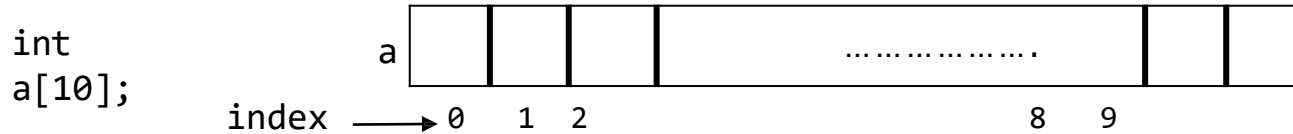
+91 9231093219 Date: **dd/mm/yyyy**



Country code Mobile no

Why Data Structures?

○ What is an Array?



Array is a data structure used to store homogeneous elements at **consecutive memory locations**.

1. Is it possible to store element at the location of a[9] (at the end)?

YES

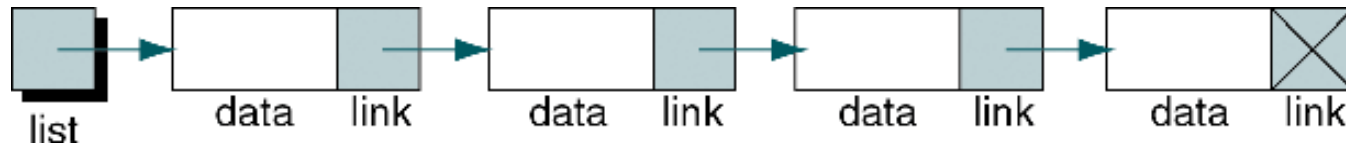
2. Is it possible to insert the elements at the middle or at the beginning locations of the array?

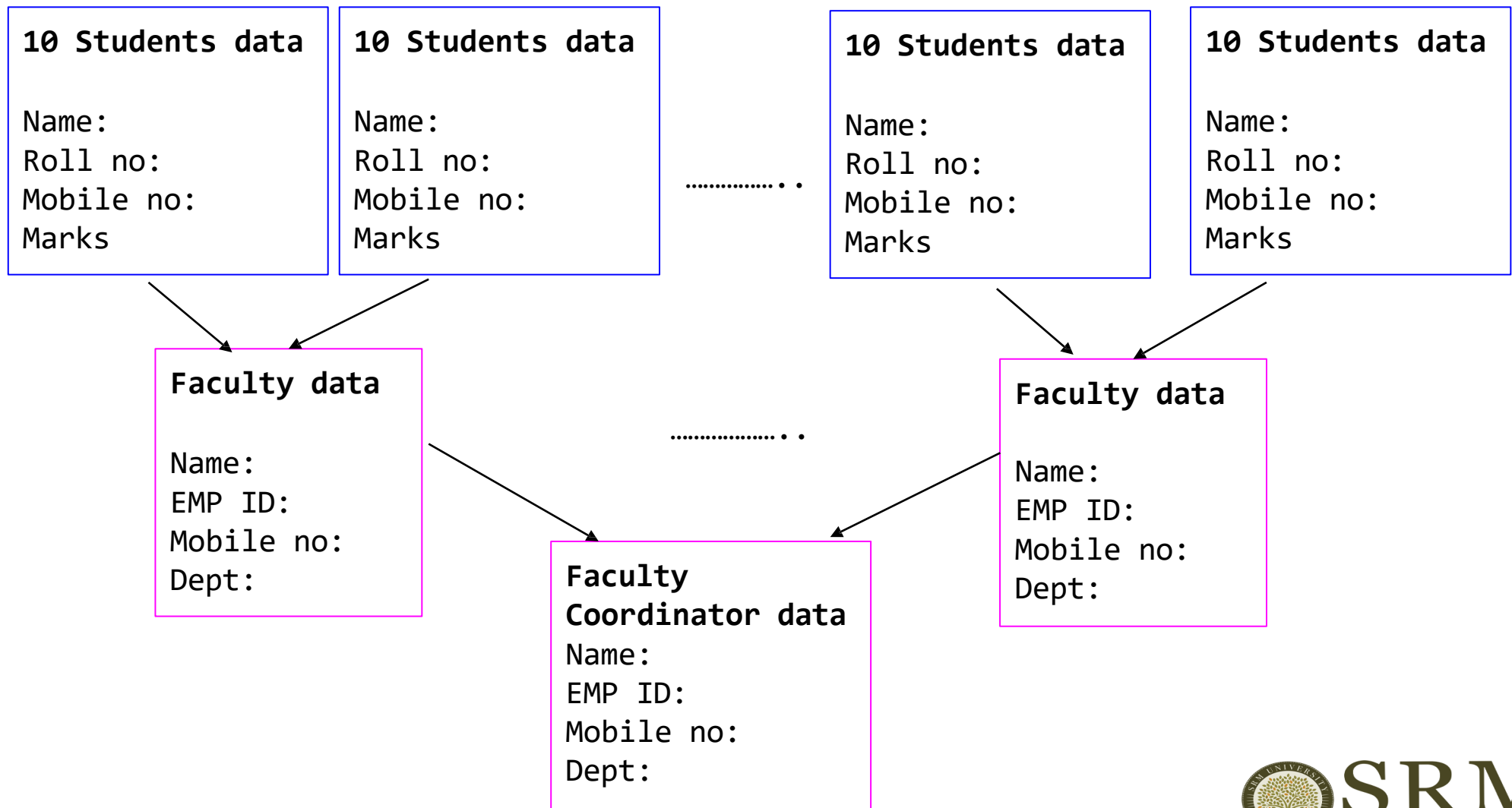
NO

3. Is it possible to store the elements into the array beyond the size of the array?

NO

If array is stored as **Linked List**





Why Data Structures?

- ▶ To **store**, **retrieve** and **process** the large amounts of complex data efficiently.
- ▶ To design **efficient algorithms** (programs).
- ▶ Programmers can save **time and memory space** in storing, accessing and processing large amounts of data.

✓ **Efficiency**

✓ **Reusability**

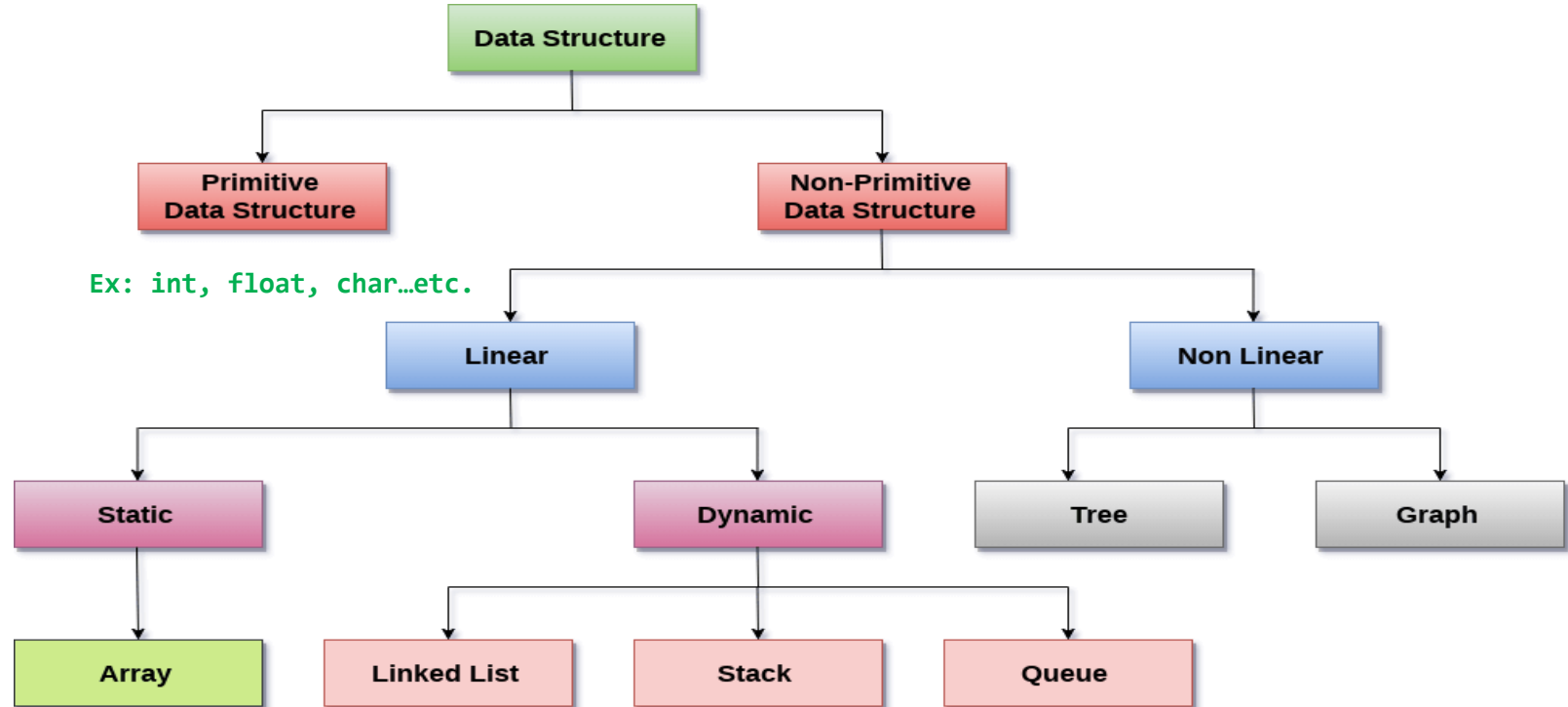
✓ **Abstraction**

What is Data Structure?

- ▶ A data structure is an aggregation of atomic and composite data into a set with the defined relationships.

OR

- ▶ A data structure is a particular way of organizing data in a computer so that it can be used effectively.



Abstract Data Types

We got answers for following questions

What is Data?

What is Data Type?

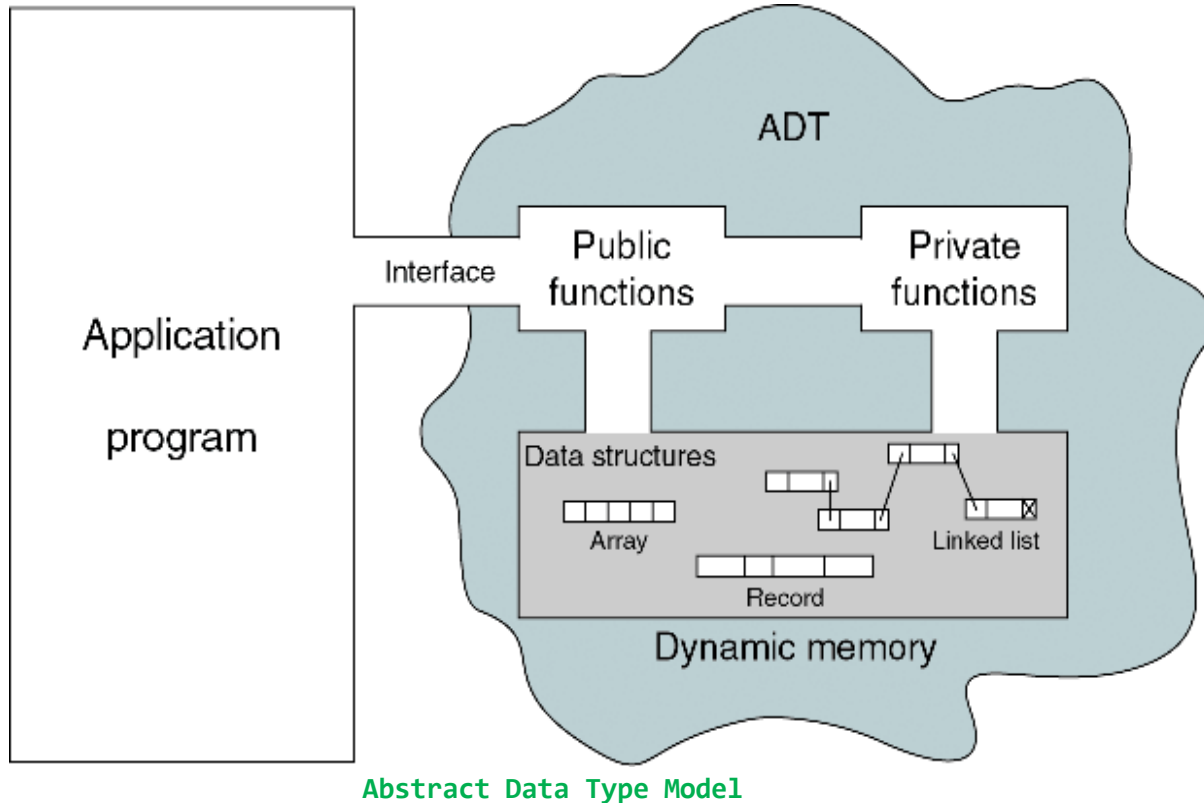
What is Data Structure?

Why Abstract Data Types?

1. Can I retrieve the elements from the array in last in first out (**LIFO**) or first in last out manner?
 2. Can I retrieve the first stored element first (first in first out (**FIFO**)) manner?
 3. Can I store the data of DATE in the format dd/mm/yyyy?
 - Can I do addition, subtraction operations on date data
 - Can I add some days?
 - Can I find the day for the specified date?
- Can a programmer create his own data types (user defined data types) for solving above type of problems?

What is Abstract Data Type?

An abstract data type is a data declaration packaged together with the operations that are meaningful for the data type.



➤ The concept **Abstraction** means:

- We know what a data type can do
- How it is done is hidden (i.e., implementation details are hidden)

➤ Programmer **encapsulates** the data and operations on the data and hides the implementation details.

ADT implementations

There are two basic structures we can use to implement an ADT:

1. Array implementations
2. Linked list implementations

Algorithm Efficiency

► Why Algorithms?

- Provides a design to solve a problem.
- Improves the efficiency in problem solving
- To utilize the resources efficiently in problem solving

► What is an Algorithm?

An algorithm is finite set of well-defined instructions, if followed, accomplishes a particular task. An Algorithm must satisfy the following criteria.

- Input
- Output
- Definiteness
- Finiteness
- Effectiveness

► Pseudocode

We can define or describe an algorithm in two ways:

1. Pseudocode
2. Flowchart

- Pseudocode is an English like representation of the algorithm logic. It is part English, and part structured code (syntax).

Note: In this Data Structures course, we use the C programming language syntax in designing pseudocodes.

Pseudocode Example

Algorithm name

Algorithm Max(*a*, *n*)

Algorithm header

```
//Algorithm used to find the maximum element in the given array a  
of size n
```

Comments

1. **begin** // Beginning of the algorithm

2. *max* = *a*[1];

3. **for** *i* = 2 to *n* do

4. **if** *a*[*i*] > *max* then

5. *max* = *a*[*i*];

6. **end if**

7. **end for**

8. **return** *max*

9. **end** // End of the algorithm

'{' can be used to begin

Loop
statement

Conditional
statement

'}' can be used to end

Statement number

a, *i*, and *max* are variables

Need of analyzing efficiency of the Algorithm

```
for i=2 to n-1 do
    if i divides n
        n is not prime
```

```
for i=2 to  $\sqrt{n}$  do
    if i divides n
        n is not prime
```

If division operation takes 1 msec time

Loop executes $n-2$ times

If $n=12$, loop executes 10 times $\rightarrow 10$ msec

If $n=102$, loop executes 100 times $\rightarrow 100$ msec

If $n=1002$, loop executes 1000 times $\rightarrow 1000$ msec = 1 sec

$n=1000002$, loop executes 1000000 times $\rightarrow 1000 \times 1000$
 $= 1000$ sec
 $= 1000/60 = 16.6$ min

Loop executes $\sqrt{n}-1$ times

If $n=12$, loop executes 3 times $\rightarrow 3$ msec

If $n=102$, loop executes 10 times $\rightarrow 10$ msec

If $n=1002$, loop executes 31 times $\rightarrow 31$ msec

$n=1000002$, loop executes 1000 times $\rightarrow 1000$ msec
 $= 1$ sec

(Values are approximate results)

Big-O Notation:

We don not need to determine the complete measure of the efficiency, only the factor that determines the magnitude is considered.

The factor is denoted with **big-0**, and read it as “in the order of.”

Algorithm (Efficiency) Analysis ↔ Big-O Analysis

- ▶ **Time complexity** of an algorithm is the amount of computer time it needs to run to completion.
- ▶ **Space complexity** of an algorithm is the amount of computer memory it needs to run to completion.

Time Complexity:

1. Linear Complexity

```
for(i=1; i<=1000; i++)  
{  
    -----  
    ----- Code  
}
```

Loop executes 1000 times

```
for(i=1; i<=n; i++)  
{  
    -----  
    ----- Code  
}
```

Loop executes n times

Complexity is $O(n)$ (in the order of n)

```
for(i=1; i<=1000; i+=2)  
{  
    -----  
    ----- Code  
}
```

Loop executes 500 times

```
for(i=1; i<=n; i+=2)  
{  
    -----  
    ----- Code  
}
```

Loop executes $n/2$ times

Complexity is $O(n/2)=O(n)$

Time Complexity:

2. Logarithmic Complexity

```
for(i=1; i<=1000; i*=2)
{
    ----- Code
}

```

i=1, iteration=1
 i=2, iteration=2
 i=4, iteration=3
 i=8, iteration=4
 i=16, iteration=5
 :
 :
 i=512, iteration=10
 i=1024, Exit

$$2^{10} \leq 1000$$

$$\log 1000 = \log 2^{10} = 10$$

```
for(i=1000; i>=1; i/=2)
{
    ----- Code
}

```

i=1000, iteration=1
 i=500, iteration=2
 i=250, iteration=3
 i=125, iteration=4
 i=62, iteration=5
 :
 :
 i=1, iteration=10
 i=0, Exit

$$1000 / 2^{10} \geq 1$$

$$\log 1000 = \log 2^{10} = 10$$

```
for(i=1; i<=n; i*=2)
{
    ----- Code
}

```

Complexity= **$O(\log n)$**

```
for(i=n; i>=1; i/=2)
{
    ----- Code
}

```

Time Complexity:

3. Quadratic Complexity

```
for(i=1; i<=n; i++)
{
    for(j=1; j<=n; j++)
    {
        ----- Code
    }
}
```

Code executes $n*n$ times

Time Complexity = $O(n^2)$

```
for (i = 1; i <=n; i++)
{
    for (j = 1; j <=i; j++)
    {
        ----- Code
    }
}
```

$i=1, j=1$, code executes=1 time

$i=2, j=1,2$ ----→2 times

$i=3, j=1,2, 3$ ----→2 times

:

:

$i=n, j=1,2, 3...n$ ----→ n times

Total no. of times = $1 + 2 + 3 + \dots + n$
 $= \frac{n(n+1)}{2}$
 $= \frac{(n^2+n)}{2}$
 $= \frac{n^2}{2} + \frac{n}{2}$

Time Complexity = $O(n^2)$

Time Complexity:

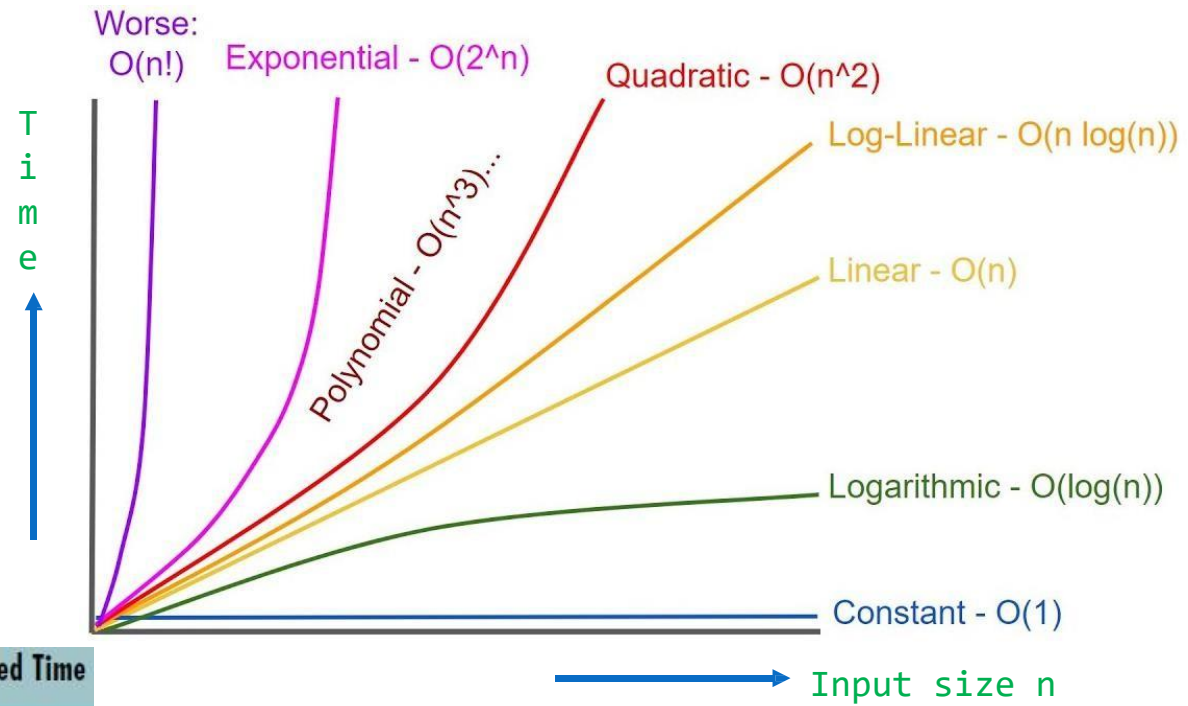
4. Linear logarithmic Complexity

```
for(i=1; i<=n; i++)  
{  
    for(j=1; j<=n; j*=2)  
    {  
        -----  
        ----- Code  
    }  
}
```

Time Complexity = $O(n \log n)$

Time Complexity:

Practical Time Complexities



Efficiency	Big-O	Iterations	Estimated Time
Logarithmic	$O(\log n)$	14	microseconds
Linear	$O(n)$	10,000	seconds
Linear logarithmic	$O(n \log n)$	140,000	seconds
Quadratic	$O(n^2)$	$10,000^2$	minutes
Polynomial	$O(n^k)$	$10,000^k$	hours
Exponential	$O(c^n)$	$2^{10,000}$	intractable
Factorial	$O(n!)$	$10,000!$	intractable

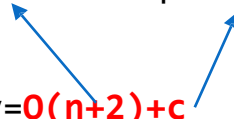
► Space Complexity:

```
Algorithm Sum(a, n)
// To find the sum of array elements
1. begin
2.     sum=0.0;
3.     for i=1 to n do
4.         sum=sum+a[i];
5.     end for
6.     return sum;
7. end
```

Variable **i** takes 1 space
Variable **sum** takes 1 space
Variable **a** takes n spaces

Total space= n+2 spaces + Space for the code

Space complexity= $O(n+2)+c$
 $=O(n)$



Space complexity=Fixed part+ Variable part



Space for the code Depends on variation in problem

*Important Note

Types of complexities:

1. Worst case (Big O)
 2. Best case
 3. Average
- (will be discussed in future semesters)

Array ADT

- A matrix is a two-dimensional data structure consisting of m rows and n columns, with a total of $m \times n$ values.
- 2D-Arrays are used to represent matrices.
- A sparse matrix is one in which the majority of the elements have 0 (zero) value.

► Why would you want to use a new representation?

- To save the storage space.
- To reduce the computing time.

► Sparse Matrix can be represented in two ways:

- Array representation.
- Linked list representation.

`a[0][2]=9,`
`a[1][1]=1,`
`a[2][0]=5,`
`a[2][2]=9,`
`a[3][0]=8,`
`a[3][3]=6`

Example

	0	1	2	3
0	0	0	9	0
1	0	1	0	0
2	5	0	9	0
3	8	0	0	6

Sparse Matrix

A matrix contains zero values more than half of its size.
(if (no.of zeros > Size(Array)/2))

Sparse Matrix using Array ADT

	0	1	2	3
0	0	0	9	0
1	0	1	0	0
2	5	0	9	0
3	8	0	0	6

Sparse Matrix:a

First row contains
no. of rows and cols
in the matrix

We store the non-zero elements
with
triple: <row, column, value>.

Sparse Matrix representation:b

	0	1	2
row	col	value	
0	4	4	6
1	0	2	9
2	1	1	1
3	2	0	5
4	2	2	9
5	3	0	8
6	3	3	6

Total no. of
non-Zero values

Row indices

Col indices

non-Zero values

Operations performed on Sparse matrices

1. Creation
2. Transpose
3. Addition
4. Multiplication

Algorithm for representing Sparse Matrix

Sparse matrix: a

	0	1	2	3
0	0	0	9	0
1	0	1	0	0
2	5	0	9	0
3	8	0	0	6

Sparse matrix representation: b
row size varies but col size fixed(=3)

	0	1	2
	row	col	value
0	4	4	6
1	0	2	9
2	1	1	1
3	2	0	5
4	2	2	9
5	3	0	8
6	3	3	6

Algorithm SparseM(a, row, col, b)
// Algorithm to represent Sparse matrix

begin

```
b[0][0] = row;
b[0][1] = col;
k = 1;
for i = 0 to row do
    for j = 0 to col do
        if (a[i][j] != 0) then
            b[k][0] = i;
            b[k][1] = j;
            b[k][2] = a[i][j];
            k++;
        end if
    end for
end for
b[0][2]=k-1;
return b;
end
```

Algorithm to Transpose a Sparse Matrix

Original matrix

row	col	val
4	4	5
1	2	10
1	4	12
3	3	5
4	1	15
4	2	12

transpose

Transposed matrix

row	col	val
4	4	5
2	1	10
4	1	12
3	3	5
1	4	15
2	4	12

```
for each row i
    take element <i, j, value> and
    store it as element <j, i, value> in the
    transpose
```

Required transposed matrix

transpose

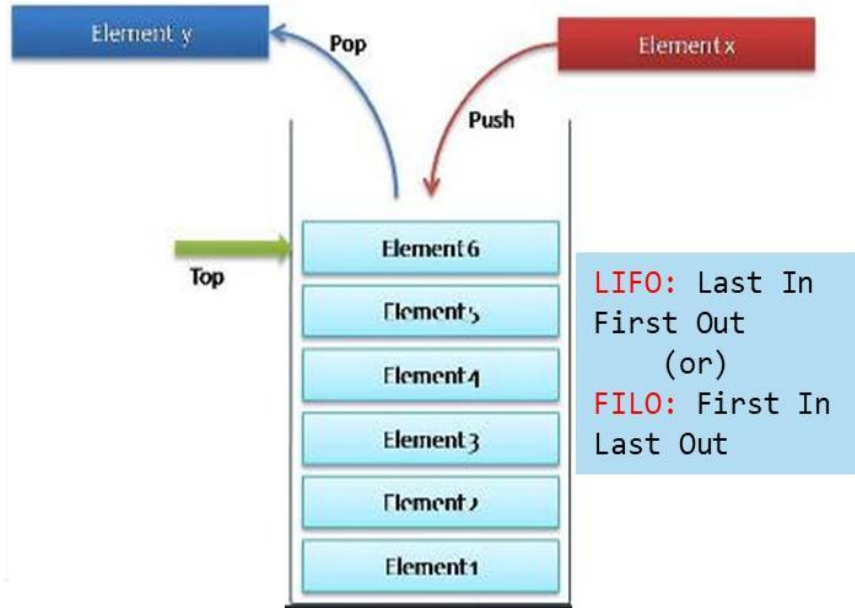
row	col	val
4	4	5
1	4	15
2	1	10
2	4	12
3	3	5
4	1	12

```
for all elements in j
    place element <i, j, value> as
    element <j, i, value> in the transpose
```

Stack Data Structure

Stacks

- Stack is a linear data structure in which items can be **inserted and removed only at one end, called the top**.
- The last item inserted into stack, is the first item to be taken out from the stack.

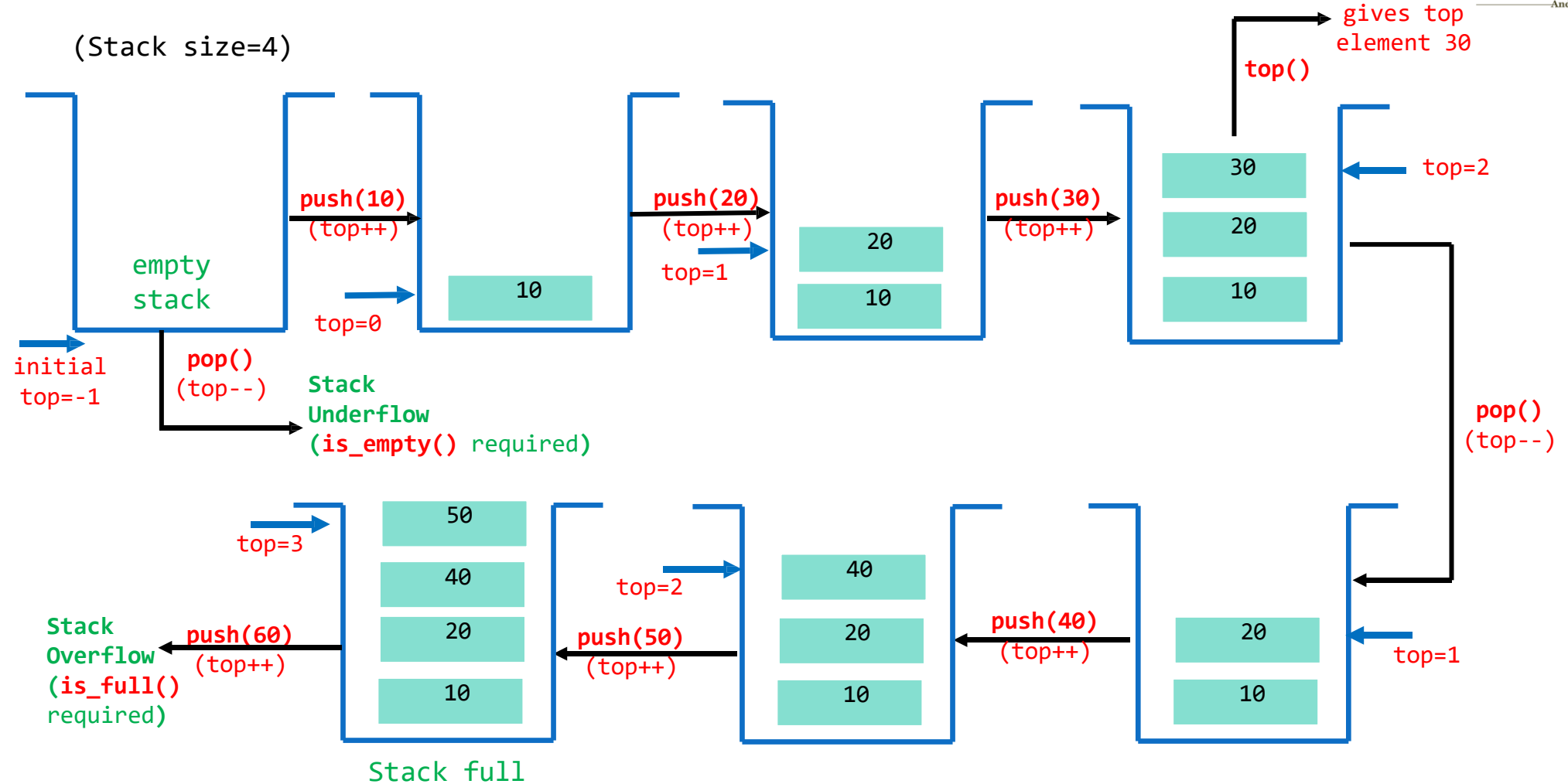


➤ Operations on Stack

- **push(data)**: adds an element to the top of the stack
- **pop()**: removes an element from the top of the stack
- **peek() or top()**: returns a copy of the element on the top of the stack without removing it.
- **is_empty()**: checks whether a stack is empty.
- **is_full()**: checks whether a stack is at maximum capacity **when stored in a static (fixed-size) structure**.

Operations on Stack (Array implementation)

(Stack size=4)

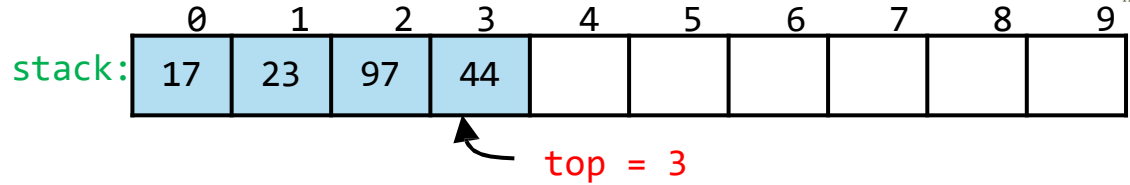


Stack implementation

Stacks can be implemented in two ways:

1. Arrays.

- ▶ Easy to implement, but the size of the array is fixed (static)
- ▶ Extra memory is not needed for pointer variables.
- ▶ It is not dynamic. It doesn't grow and shrink depending on needs at runtime.
- ▶ Performing operation **is_full()** is required.



2. Linked lists.

- ▶ This implementation of a stack can grow and shrink according to the needs at runtime.
- ▶ Extra memory is needed for pointer variables.

Pseudocodes for Stack Operations

```
MAX_SIZE = 100  
ARRAY stack[MAX_SIZE]  
top = -1;
```

Algorithm: `is_full(top)`

begin

```
    if (top == MAX_SIZE - 1) then  
        return True;  
    else  
        return False;  
    end if
```

end

Algorithm: `push(stack, top, data)`

begin

```
    if (is_full(top) == True) then  
        print("Stack is full");  
    else  
        top = top + 1;  
        stack[top] = data;  
    end if  
    return top;
```

end

Algorithm: `is_empty(top)`

begin

```
    if (top == -1) then  
        return True;  
    else  
        return False;  
    end if
```

end

Algorithm: `pop(stack, top)`

begin

```
    if (is_empty(top) == True) then  
        print("Stack is empty");  
    else  
        popped_item = stack[top];  
        top = top - 1;  
    end if  
    return (popped_item, top);
```

end

Pseudocodes for Stack Operations

```
Algorithm: peek(stack, top)
begin
    if (is_empty(top) == True) then
        print("Stack is empty");
    else
        peeked_item = stack[top];
    end if
    return peeked_item;
end
```

Time Complexities of operations on stack:

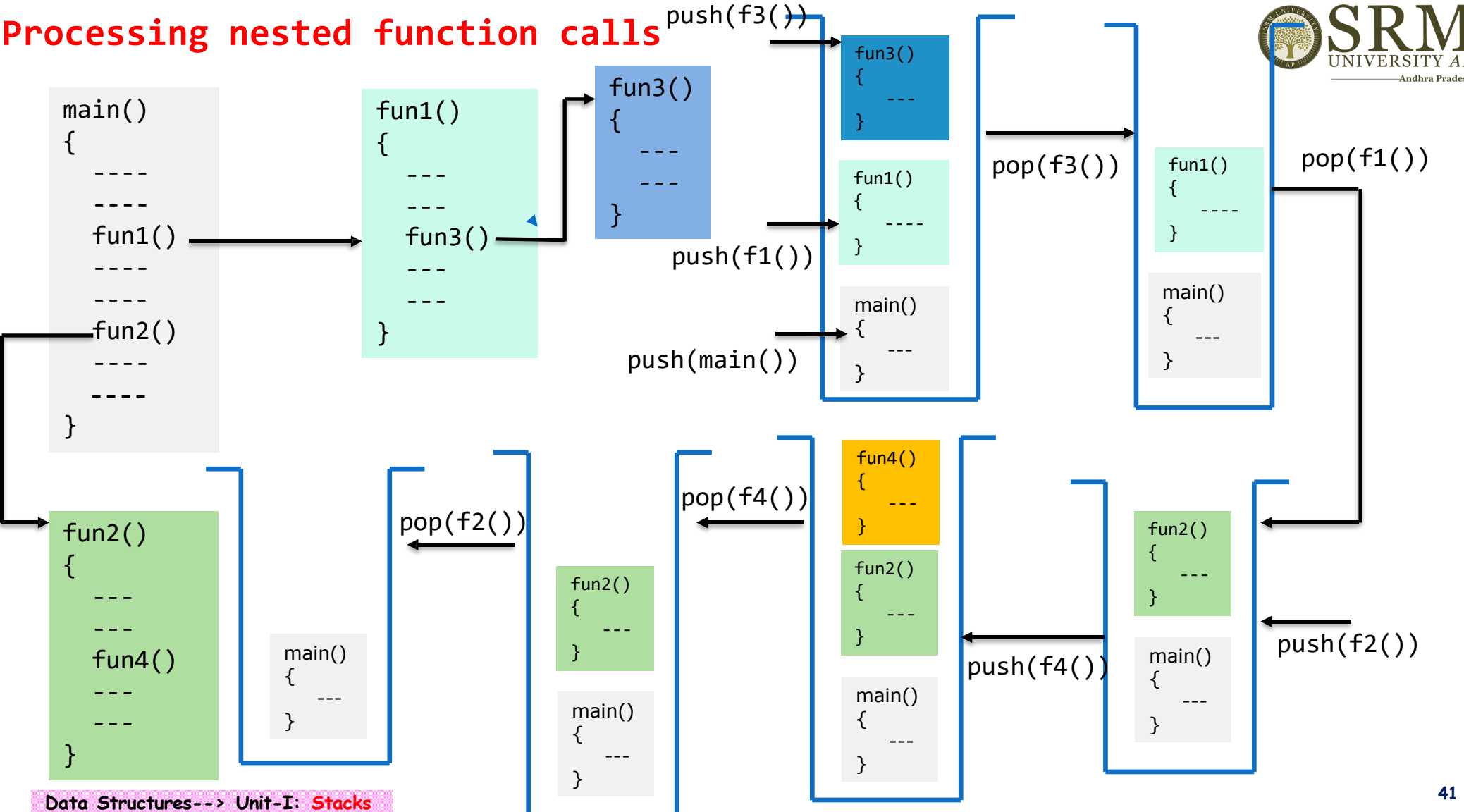
Operation	Time Complexity
1. is_full()	O(1)
2. is_empty()	O(1)
3. peek() or top()	O(1)
4. push(data)	O(1)
5. pop()	O(1)

- These all operations do not contain any loop.

Applications of Stacks

- ▶ Arithmetic Expression Evaluation.
 - ▶ Expression Conversion.
 - ▶ Processing functions call.
- 
- Infix expression to Prefix expression.
 - Infix expression to Postfix expression.
 - Evaluating postfix expression.
 - Evaluating prefix expression.
- ▶ Processing recursive calls (Recursion).
 - ▶ Backtracking: To track the recursive calls in building optimal solutions.
 - ▶ Syntax Parsing (in Compilers).
 - ▶ Memory Management(in Operating Systems).
 - ▶ Parenthesis Checking.
 - ▶ String Reversal.
 - ▶ Text Editors: Undo() or Redo () mechanism (Excel, Notepad or WordPad etc.)
 - ▶ Backward() and forward() buttons in browser.

Processing nested function calls



Processing recursive calls (Recursion)

```
#include<stdio.h>
```

```
int factorial(int n);
```

```
int main()
```

```
{
```

```
    int n;
```

```
    printf("Enter a positive integer: ");
```

```
    scanf("%d",&n);
```

```
    printf("Factorial of %d = %d", n, factorial(n));
```

```
}
```

```
int factorial(int n)
```

```
{
```

```
    if (n>=1)
```

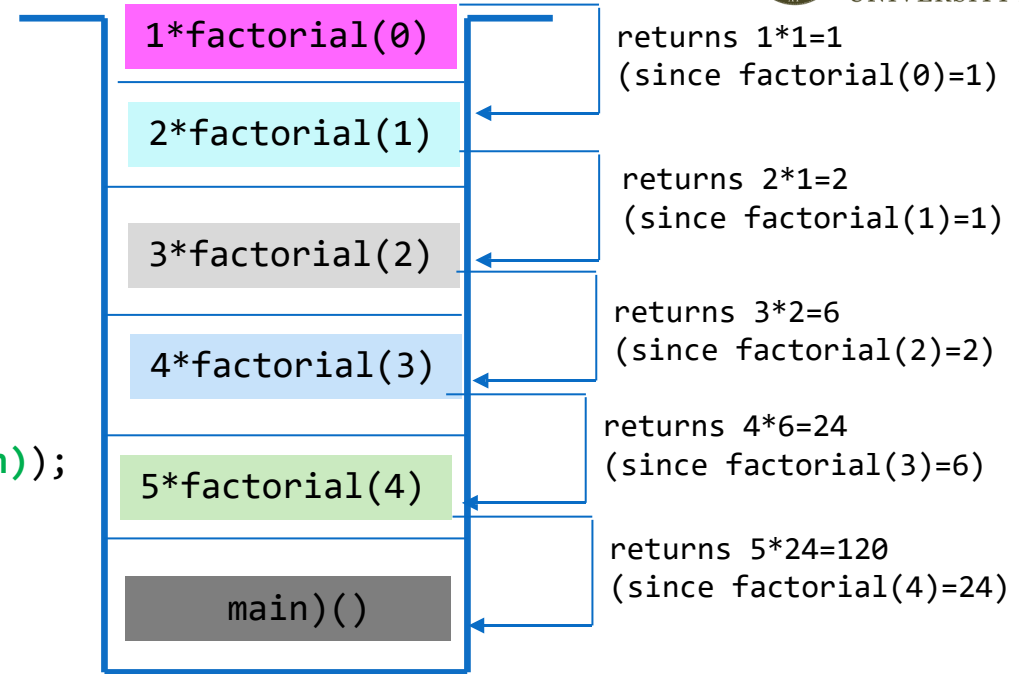
```
        return n*factorial(n-1);
```

```
    else
```

```
        return 1;
```

```
}
```

Example: n=5



main() function prints **Factorial of 5=120**

Expression Representation

Popular methods used for representation of an expression →

Infix	$x + y$	Operator between operands.
Prefix	$+ x y$	Operator before operands.
Postfix	$x y +$	Operator after operands.

Precedence/Associativity table

Operators	Associativity	Precedence
$\wedge$ exponentiation	Right to left	Highest followed by *Multiplication and /division
* Multiplication, / division	Left to right	Highest followed by + addition and - subtraction
+ addition, - subtraction	Left to right	lowest

Converting Infix to Postfix

Algorithm for converting Infix to Postfix

Step 1: Consider the next element in the input.

Step 2: If it is operand, display it.

Step 3: If it is opening parenthesis, insert it on stack.

Step 4: If it is an operator, then

- If stack is empty, insert operator on stack.
- If the top of stack is opening parenthesis, insert the operator on stack
- If it has higher priority than the top of stack, insert the operator on stack.
- Else, delete the operator from the stack and display it, repeat Step 4.

Step 5: If it is a closing parenthesis, delete the operator from stack and display them until an opening parenthesis is encountered. Delete and discard the opening parenthesis.

Step 6: If there is more input, go to Step 1.

Step 7: If there is no more input, delete the remaining operators to output.

Example-1: $3*3/(4-1)+6*2$

Expression	Stack	Output
3	Empty	3
*	*	3
3	*	33
/	/	33*
(	/(	33*
4	/(	33*4
-	/(-	33*4
1	/(-	33*41
)	/	33*41-
+	+	33*41- /
6	+	33*41- /6
*	+	33*41- /6
2	+	33*41- /62
	Empty	33*41- /62*+

Converting Infix to Postfix

Example-1: $3 * 3 / (4 - 1) + 6 * 2$

Example-2: $(a*b) + (d-c)$

$ab^* + dc^-$

ab^*dc^-+

Example-3: $a + b / c + d * (e-f) ^ g$

$a + b / c + d * ef^- ^g$

$a + b / c + d * ef^-g^$

$a + bc/ + d * ef^-g^$

$a + bc/ + def^-g^{*}$

$abc/+ + def^-g^{*}$

$abc/+def^-g^{*}+$

Example-4: $K + L - M*N + (O^P) * W/U/V * T + Q$

Converting Infix to Prefix

Algorithm for Infix to Prefix Conversion:

Step 1: Insert “)” onto stack, and add “(” to end of the expression .

Step 2: Scan expression from right to left and repeat Step 3 to 6 for each element of A until the stack is empty .

Step 3: If an operand is encountered, add it to output .

Step 4: If a right parenthesis is encountered, insert it onto stack .

Step 5: If an operator is encountered then,
 a. Delete from stack and add to output (each operator on the top of stack) which has same or higher precedence than the operator.
 b. Add operator to stack.

Step 6: If left parenthesis is encountered then ,
 a. Delete from the stack and add to B (each operator on top of stack until a left parenthesis is encountered).
 b. Remove the left parenthesis.

Step 7: Exit

Converting Infix to Prefix

Example-1: $(a*b) + (d-c)$

Example-2: $a + b / c + d * (e-f) ^ g$

Example-3: $K + L - M*N + (O^P) * W/U/V * T + Q$

Postfix expression evaluation

Algorithm for evaluating postfix expression

Step-1: Read all the symbols one by one from left to right in the given Postfix Expression

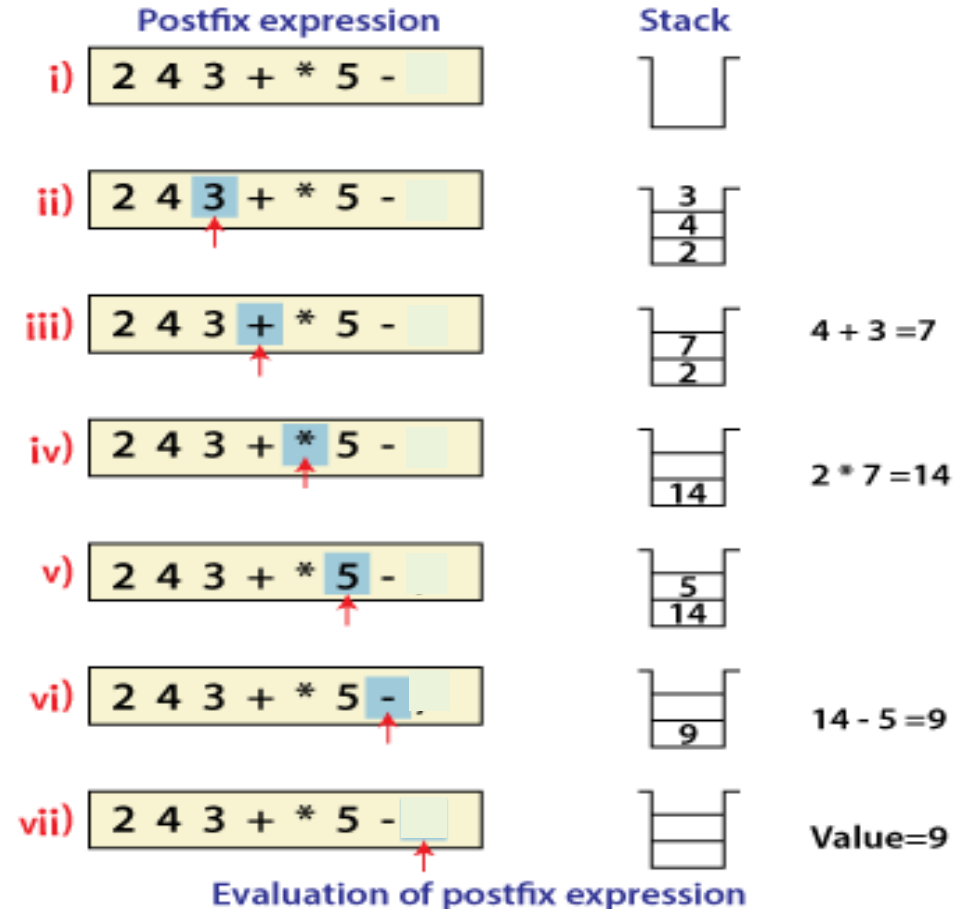
Step-2: If the reading symbol is **operand**, then push it on to the Stack.

Step-3: If the reading symbol is **operator** (+ , - , * , / etc.), then perform **TWO** pop operations and store the two popped operands in two different variables (operand1 and operand2). Then perform reading symbol operation using operand1 and operand2 and push result back on to the Stack.

Step-4: Finally! perform a pop operation and display the popped value as final result.

Example infix expression: $2 * (4 + 3) - 5$

Equivalent postfix: $2\ 4\ 3\ +\ *\ 5\ -$



Postfix expression evaluation

Example infix expression: $2 * (4 + 3) - 5$

Equivalent postfix: $2\ 4\ 3\ +\ *\ 5\ -$

Input	Stack	Operation
2	2	
4	2 4	
3	2 4 3	
+	2 2 7	$4+3=7$
*	empty 14	$2*7=14$
5	14 5	
-	empty 9	$14-5=9$
No i/p	empty	9

Prefix expression evaluation

Example infix expression: $2 * (4+3) - 5$

Equivalent prefix: $- 5 * 2 + 4 3$

Reverse of prefix: $3 4 + 2 * 5 -$

Reverse of prefix: $3 4 + 2 * 5 -$

Above expression is evaluated according the algorithm of evaluating postfix expressions.

Note: Prefix notation also known as Polish Notation or Warsaw notation, and Postfix notation is also known as Reverse Polish notation

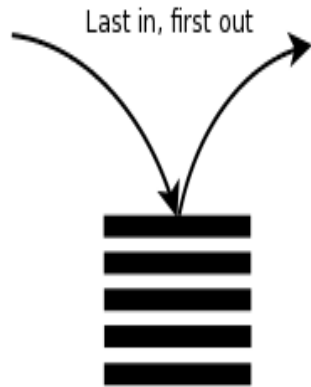
Queue Data Structure

Queues

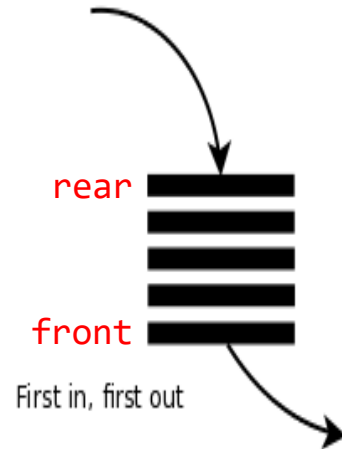
- **Queue** is a linear data structure, where a data item insertion is done at one end (called **rear**) and deletion is performed at the other end (called **front**).
- Accessing the elements of queue follows a **First In, First Out (FIFO)** order.



Stack:



Queue:



➤ Operations on Queue

- **enqueue(data)**: This is used to add elements into the queue at the rear (back) end.
- **dequeue()**: removes an element from the queue at the front end of the queue.
- **is_empty()**: checks whether a queue is empty.
- **is_full()**: checks whether a queue is at maximum capacity **when stored in a static (fixed-size) structure**.

Queue implementation

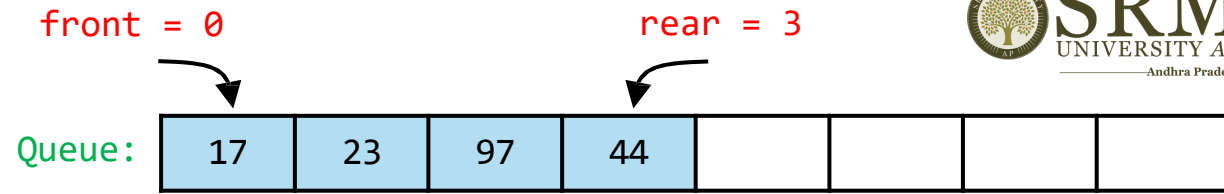
Queues can be implemented in two ways:

1. Arrays.

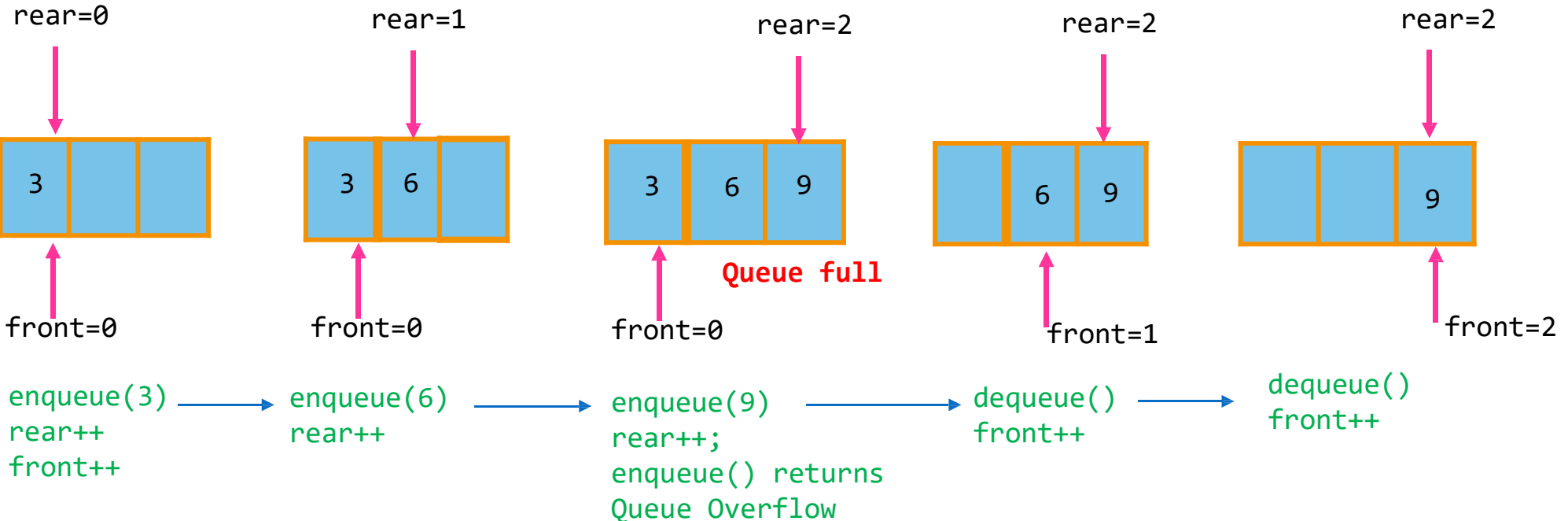
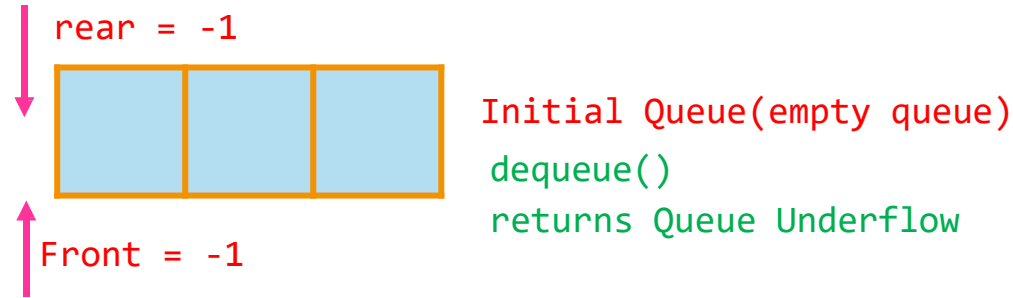
- ▶ Easy to implement, but the size of the array is fixed (static)
- ▶ Extra memory is not needed for pointer variables.
- ▶ It is not dynamic. It doesn't grow and shrink depending on needs at runtime.
- ▶ Performing operation **is_full()** is required.

2. Linked lists.

- ▶ This implementation of a stack can grow and shrink according to the needs at runtime.
- ▶ Extra memory is needed for pointer variables.



Operations on Queues (Array implementation)



Array implementation of Queue

```
#define SIZE 20
void enqueue(int);
void dequeue();
void read();
int queue[SIZE], front = -1, rear = -1;
```

```
void enqueue(int value)
{
    if(rear == SIZE-1)
        printf("\n Queue is full");
    else
    {
        if(front == -1)
            front = 0;
        rear++;
        queue[rear] = value;
        printf("\n Successfully inserted!!!");
    }
}
```

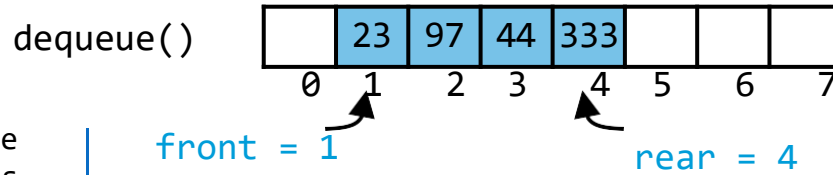
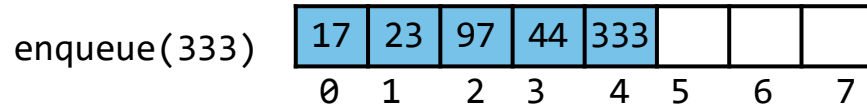
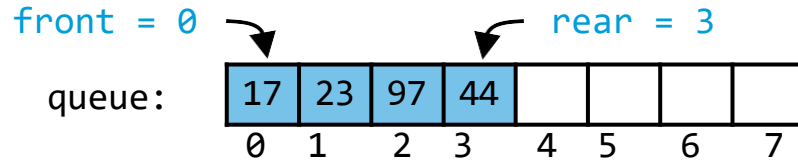
```
void dequeue()
```

```
{
    if(front == -1)
        printf("\nQueue is empty");
    else
    {
        printf("Successfully deleted\n");
        printf("Deleted ele : %d", queue[front]);
        front++;
        if(front == SIZE)
            front = rear = -1;
    }
}
```

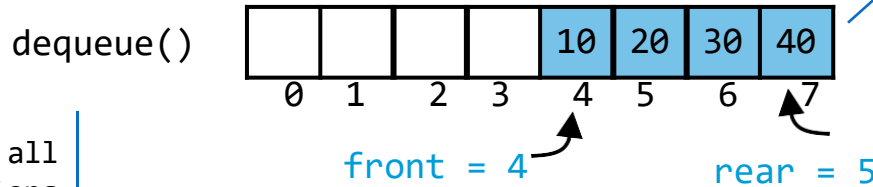
```
void read()
```

```
{
    if(rear == -1)
        printf("\nQueue is Empty!!!");
    else
    {
        int i;
        printf("\nQueue elements are:\n");
        for(i=front; i<=rear; i++)
            printf("%d\t", queue[i]);
    }
}
```

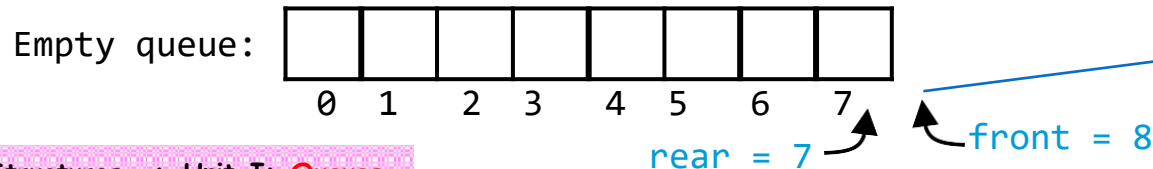
Problems with Linear queues



After some
insertions
and deletions



After all
deletions

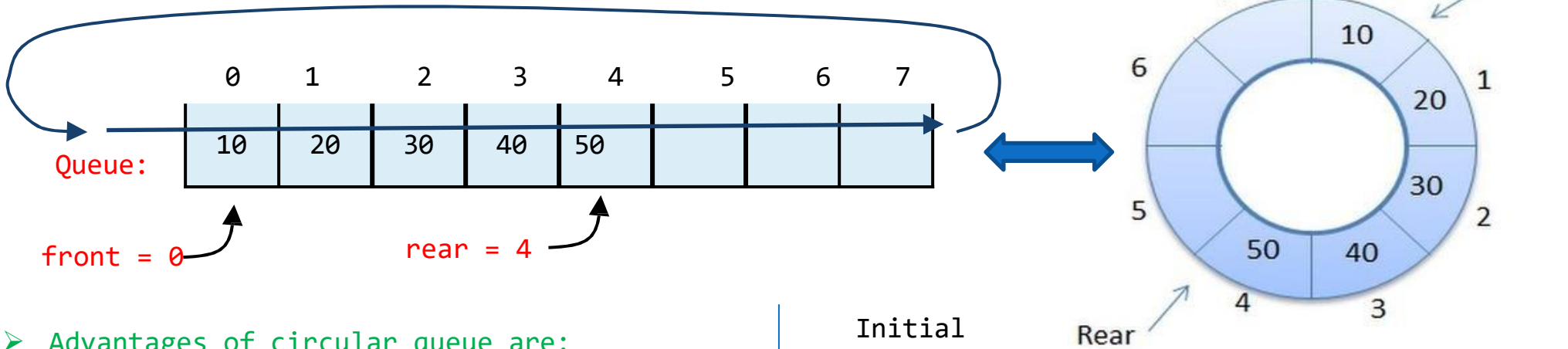


- If the **rear** of the queue is at the end of the array (the **highest index**).
- Even if there are empty cells at the beginning of the array, **rear** can't go any further.
- Because all the elements are removed, the **rear** points to end of queue (the **highest index**).
- Hence a new element can not be inserted.

**if front=SIZE of array then
rear=front=-1;**

Circular queues

An array holding the queue elements as circular (joined at the ends)



➤ Advantages of circular queue are:

- **Easier for insertion-deletion:** Elements can be inserted easily if there are vacant locations.
- **Efficient utilization of memory:** No wastage of memory as it uses the unoccupied space.

Initial

$\text{rear} = -1, \text{front} = -1$

(when Queue is empty)

- Elements were added to this queue in the order 10, 20, 30, 40, 50, and will be removed in the same order
 - Index of $\text{front} = (\text{front} + 1) \% \text{Queue.length};$
 - Index of $\text{rear} = (\text{rear} + 1) \% \text{Queue.length};$

- ▶ When a resource is shared among multiple consumers.

Examples: CPU scheduling, Disk Scheduling.

- ▶ When data is transferred asynchronously (data not necessarily received at same rate as sent) between two processes.

Examples: IO Buffers, pipes, file IO, printer queue etc.

- ▶ Data packets waiting to be transmitted over the Internet.
- ▶ Queue used to model real-world situations.

Examples: People waiting in line at a bank, airplanes waiting to take off, stores and reservation centres,..etc.